

CS336 作业 4 (数据): 语言建模数据的过滤

版本 26.0.1

CS336 课程团队

2026 年春季

1 作业概述

在本次作业中，你将通过动手实践，学习如何过滤网页抓取数据，为语言建模构建数据集。

你将实现的内容。

1. 将 Common Crawl 中的 HTML 转换为文本。
2. 使用多种方法过滤提取出的文本（例如，处理有害内容、个人可识别信息等）。
3. 对训练数据进行去重。

你将运行的实验。

1. 在不同数据集上训练语言模型，以更好地理解特定数据处理决策对模型性能的影响。

代码的组织方式。

本次作业的全部代码以及这份说明文档均可在 GitHub 上找到：

github.com/stanford-cs336/assignment4-data

请使用 `git clone` 克隆仓库。如果有更新，我们会通知你；届时可以使用 `git pull` 获取最新内容。

1. `cs336_basics/*`: 该文件夹包含训练作业 1 中所构建模型的代码。我们对模型做了少量优化，将作业 1 中手工实现的一些组件替换为对应的 PyTorch 原生实现（例如，使用 PyTorch 内置的交叉熵计算内核）。此外，我们还提供了支持多 GPU 分布式数据并行训练的训练脚本。你将使用该脚本，在经过自己过滤的数据上训练模型。
2. `cs336_data/*`: 你将在这里编写作业 4 的代码。我们创建了一个名为 `cs336_data` 的模块，其中包含本次作业的起始代码和占位代码。
3. `tests/*.py`: 其中包含所有必须通过的测试。这些测试会调用 `tests/adapters.py` 中定义的接口钩子。你需要实现适配器，将自己的代码连接到测试。编写更多测试和/或修改测试代码可能有助于调试，但你的实现应当能够通过最初提供的原始测试套件。
4. `README.md`: 该文件更详细地介绍了预期的目录结构，也提供了环境配置的基本说明。

如何提交。

你需要向 Gradescope 提交以下文件：

- `writeup.pdf`: 回答所有书面问题。请对答案进行排版。
- `code.zip`: 包含你编写的全部代码。

2 过滤 Common Crawl

尽管大型语言模型主要使用互联网数据训练，但大多数研究者并不会自行开发网络爬虫来获取模型的训练数据，而是使用公开的网页抓取数据。最常用的公开网页抓取数据来自 Common Crawl。这家非营利组织提供免费的网页语料库，包含“跨越 17 年的超过 2500 亿个网页”。¹

不过，正如我们将看到的，要把 Common Crawl (CC) 数据转储转换为可用于语言模型训练的数据，需要做大量工作。例如，网页的原始数据是 HTML，而我们希望从中提取文本。此外，许多网页可能质量较低、与其他网页完全或近乎重复、含有有害内容，或者包含敏感信息；我们可能希望从数据集中滤除这些网页，或删除其中不需要的内容。在本次作业中，我们将搭建一条流水线，执行上述若干步骤，把原始互联网数据转换为可用的语言模型训练集。

2.1 查看数据

在着手实现任何内容之前，查看原始数据并对其形成直观认识，总是很有帮助。CC 数据以三种格式提供：

WARC

即“Web ARChive format”（网页归档格式）。这类文件包含 CC 原始数据，包括网页 ID 和 URL、元数据、HTTP 请求的详细信息（例如请求日期和时间、服务器 IP 地址），当然也包括网页的原始内容（例如 HTML）。

WAT

即“Web Archive Transformation”（网页归档转换）。这类文件包含从 WARC 文件中提取的更高层次元数据，并以 JSON 对象形式转储。例如，对于 HTML 网页，这些元数据包括该网页中的链接列表以及网页标题。

WET

即“Web Extracted Text”（网页提取文本）。这类文件包含从原始 HTML 网页中提取的纯文本。

在下面的题目中，我们将查看一个 WARC 文件以及对应的 WET 文件。这些文件来自 Common Crawl 在 2026 年的一次抓取，下载方式如下：

```
# 下载一个 WARC 示例文件。
$ wget https://data.commoncrawl.org/crawl-data/CC-MAIN-2026-12/segments/1741613651012.6/warc/CC-MAIN-20260310152245-20260310182245-00000.warc.gz
# 下载与之对应的 WET 文件。
$ wget https://data.commoncrawl.org/crawl-data/CC-MAIN-2026-12/segments/1741613651012.6/wet/CC-MAIN-20260310152245-20260310182245-00000.warc.wet.gz
```

这些文件已存放在以下位置：

- /shared-data/CC/example.warc.gz
- /shared-data/CC/example.warc.wet.gz

警告：这些文件包含完全未经筛选的互联网网页，其中可能有大量潜在有害内容。遇到不愿阅读的文档时，你可以继续向下滚动，直接跳到下一个文档。

¹<https://commoncrawl.org/>

题目 (look_at_cc): 查看 Common Crawl (4 分)

- (a) 使用上述路径提供的 WARC 文件副本。先查看该文件中的第一个网页。这是一个 gzip 压缩文件，可以使用以下命令浏览其内容：

```
$ zcat /shared-data/CC/example.warc.gz | less
```

less 允许你使用键盘方向键、Page Up 和 Page Down 浏览文件。按“q”退出。

查看最开始的那个网页。它的 URL 是什么？现在还能访问吗？仅查看原始 HTML，你能判断这个网页大概讲的是什么吗？

提交内容：用 2-3 句话回答。

- (b) 现在查看对应的 WET 文件：

```
$ zcat /shared-data/CC/example.warc.wet.gz | less
```

注意，WET 文件包含 HTTP 头部（例如 Content-Length），它们并不属于提取出的文本内容。查看第一个样本时，你会发现，它包含从刚才看到的原始 HTML 中提取出的文本。

注意，提取出的许多文本仍然带有明显的 HTML 结构痕迹，而并非网页的主要内容。在这些文本中，有没有你认为应当被提取器滤除的部分？请思考这类文本作为训练数据的质量：在这样的文本上训练模型，可能会出现什么问题？反过来，模型又可能从这个网页中提取到哪些有用信息？

提交内容：用 3-4 句话回答。

- (c) 什么样的训练样本才算好，强烈依赖于具体情境。请描述一个适合把该样本纳入训练数据的应用领域，以及一个可能不适合的应用领域。

提交内容：用 1-2 句话回答。

- (d) 再看一些样本，以便更好地了解 Common Crawl 中包含什么。继续查看另外 25 条 WET 记录。对于每条记录，简要评论文档的语言（如果能识别）、域名、网页类型等。你查看了多少个样本，才遇到一个自己认为“高质量”的网页？

提交内容：对 25 个文档分别给出简短标注，记录其语言、域名、网页类型，以及其他有关该文档的备注。报告遇到一个高质量样本之前所查看的样本数量。

2.2 从 HTML 转换为文本

通过查看上面的 WARC 和 WET 文件，你可能已经意识到，从 HTML 中提取文本并不容易。通常，任何提取流程都会寻找 HTML 中的可见内容（例如 <p> 标签，这类标签应当包含文本块）。但这种做法仍可能提取出远多于我们在浏览器中打开网页时所认为的主要内容。例如，打开 StackOverflow 时，主要内容是问题和回答；然而从技术上说，菜单选项、指向其他 StackExchange 中无关页面的链接、页脚，以及注册或登录链接，也都是可见文本。要可靠地区分这些内容与网页的主体内容，是一项挑战。

许多工具都实现了文本提取流水线。在本次作业中，我们将使用 Resiliparse² 库进行文本提取。Resiliparse 还可以帮助解决一个更基础的问题：检测包含原始内容的字节数据所采用的文本编码。虽然网页中的大多数内容都使用 UTF-8 编码（据维基百科统计，占 98.2%），但我们的文本提取流水线也应当能够稳健地处理其他编码。

注意：

我们建议使用 FastWARC 库遍历每个 WARC 文件中的记录。具体来说，下面这些类可能会用到：

```
from fastwarc.warc import ArchiveIterator, WarcRecordType
```

题目 (extract_text)：从 HTML 转换为文本 (3 分)

- (a) 编写一个函数，从包含原始 HTML 的字节串中提取文本。使用 `resiliparse.extract.html2text.extract_plain_text` 进行提取。该函数需要字符串，因此你必须先将字节串解码为 Unicode 字符串。请注意，输入字节串不一定使用 UTF-8 编码，所以当 UTF-8 解码失败时，你的函数应当能够检测其编码。Resiliparse 还提供了 `resiliparse.parse.encoding.detect_encoding()`，这个函数可能会有帮助。

提交内容：一个接收 HTML 字节串并返回提取文本字符串的函数。实现适配器 `[run_extract_text_from_html_bytes]`，并确保它通过 `uv run pytest -k test_extract_text_from_html_bytes`。

- (b) 在单个 WARC 文件上运行你的文本提取函数。将输出与对应 WET 文件中的提取文本进行比较。你观察到了哪些差异和/或相似之处？哪一种提取结果看起来更好？

提交内容：用 2–3 句话，对比你自己的函数提取出的文本与 WET 文件中的提取文本。

2.3 语言识别

互联网上的网页使用数千种语言书写。然而，在大多数计算预算下，训练一个能够大规模、有效利用如此多样数据的多语言模型是一项挑战。因此，许多从 Common Crawl 构建的语言建模训练集，只包含有限的一组语言的数据。

一个适合这项任务的库是 fastText (<https://fasttext.cc>)，它提供高效的文本分类器。这个库既提供使用自己的数据训练分类器的基础设施，也提供一组预训练模型，其中包括语言识别模型。你可以从 <https://fasttext.cc/docs/en/language-identification.html> 下载 fastText 语言识别模型 `lid.176.bin`；该模型也已放在 `/shared-data/classifiers/lid.176.bin`。

通常，语言过滤器会使用分类器给出的分数，决定是否保留某个网页。请使用 fastText 语言识别分类器实现一个语言识别过滤器，并让它给出一个非负分数，表示其对预测结果的置信程度。

²<https://resiliparse.chatnoir.eu/en/stable/index.html>

题目 (language_identification): 语言识别 (6 分)

- (a) 编写一个函数，接收 Unicode 字符串，并识别其中主要使用的语言。函数应返回一个二元组，其中包含语言标识符，以及一个介于 0 和 1 之间、表示预测置信度的分数。

提交内容：一个执行语言识别的函数，给出排名第一的语言预测及其分数。实现适配器 `[run_identify_language]`，并确保它通过 `uv run pytest -k test_identify_language` 中的两个测试。注意，这些测试假定英语和中文分别使用特定的字符串标识符“en”和“zh”；因此，如有必要，测试适配器应进行相应的标识符重映射。

- (b) 语言模型在推理时的行为，在很大程度上取决于它的训练数据。因此，数据过滤流水线中的问题可能导致下游问题。你认为，语言识别流程中的问题可能引发哪些问题？在风险更高的场景中（例如部署面向用户的产品时），你会如何缓解这些问题？

提交内容：用 2-5 句话回答。

- (c) 在从 WARC 文件中提取出的文本上运行你的语言识别系统（使用你之前实现的文本提取函数）。随机选择 20 个样本，人工识别其语言，并将自己的标签与分类器预测进行比较。报告分类器出现的任何错误。英语文档所占的比例是多少？根据你的观察，过滤时采用怎样的分类器置信度阈值比较合适？

提交内容：用 2-5 句话回答。

2.4 个人可识别信息

互联网上包含大量可用于联系或识别个人的信息，例如电子邮件地址、电话号码或 IP 地址。我们可能不希望面向用户的语言模型输出真实个人的这类信息，因此，一个常见处理步骤是在训练数据集中将这些信息遮蔽掉。

接下来，你将实现三个流程，分别遮蔽（a）电子邮件地址、（b）电话号码和（c）IP 地址。

题目 (mask_pii): 个人可识别信息 (3 分)

- (a) 编写一个遮蔽电子邮件地址的函数。函数接收字符串作为输入，将其中所有电子邮件地址替换为字符串 `"|||EMAIL_ADDRESS|||"`。为检测电子邮件地址，你可以查找能够可靠完成此任务的正则表达式。

提交内容：一个将给定字符串中所有电子邮件地址替换为 `"|||EMAIL_ADDRESS|||"` 的函数，返回由新字符串和被遮蔽实例数量组成的二元组。实现适配器 `[run_mask_emails]`，并确保它通过 `uv run pytest -k test_mask_emails` 中的全部测试。

- (b) 编写一个遮蔽电话号码的函数。函数接收字符串作为输入，将其中所有电话号码替换为字符串 `"|||PHONE_NUMBER|||"`。要可靠地完成这一任务可能极其困难，因为电话号码的书写格式可能极为多样。

不过，你应尽量覆盖至少美国最常用的电话号码格式，并对细微的语法形式偏差保持稳健。

提交内容：一个将给定字符串中的电话号码替换为 "|||PHONE_NUMBER|||" 的函数，返回由新字符串和被遮蔽实例数量组成的二元组。实现适配器 `[run_mask_phone_numbers]`，并确保它通过 `uv run pytest -k test_mask_phones`。

- (c) 编写一个遮蔽 IP 地址的函数。本题只需考虑 IPv4 地址即可（由点分隔的 4 个数字，每个数字最大为 255）。函数接收字符串作为输入，将其中所有 IP 地址替换为字符串 "|||IP_ADDRESS|||"。

提交内容：一个将给定字符串中的 IPv4 地址替换为 "|||IP_ADDRESS|||" 的函数，返回由新字符串和被遮蔽实例数量组成的二元组。实现适配器 `[run_mask_ips]`，并确保它通过 `uv run pytest -k test_mask_ips`。

- (d) 如果直接、简单地把这些过滤器应用于训练集，你认为语言模型在下游使用中可能出现哪些问题？你会如何缓解这些问题？

提交内容：用 2–5 句话回答。

- (e) 在从 WARC 文件中提取出的文本上运行你的 PII 遮蔽函数（使用你之前实现的文本提取函数）。随机查看 20 个发生过替换的样本，给出一些假阳性（误报）和假阴性（漏报）的例子。

提交内容：用 2–5 句话回答。

2.5 有害内容

未经筛选的网页数据转储包含大量我们不希望语言模型在推理时复述的文本。令人意外的是，某些这类训练样本甚至来自通常无害的网站，例如维基百科——用户在一些页面上留下的评论可能具有很强的毒性。尽管几乎不可能为“有害”划出明确边界，但许多数据过滤流水线仍会对主要包含有害内容的网页进行一定程度的过滤。

识别此类内容的方法有很多，例如统计禁用词表中词语的出现次数，或基于人工评审者提供的标签构建简单分类器。在本部分作业中，我们重点识别两大类有害内容：“不适合工作场所的内容”（Not safe for work, NSFW；包括色情、粗俗用语或其他可能令人不适的内容），以及毒性言论（“粗鲁、不尊重他人或不讲理、很可能使人退出讨论的语言”³）。我们将使用 Dolma 项目 [1] 提供的 fastText 预训练模型，判断输入文本是否属于这两类中的任意一类。这些分类器使用 Jigsaw Toxic Comments 数据集训练，该数据集包含按多种标签分类的维基百科评论。⁴

NSFW 分类器可以从以下位置下载：

`dolma-artifacts.org/.../jigsaw_fasttext_bigrams_nsfw_final.bin`

仇恨言论分类器位于：

`dolma-artifacts.org/.../jigsaw_fasttext_bigrams_hatespeech_final.bin`

我们还把这两个分类器放在了共享数据目录中：

³<https://current.withgoogle.com/the-current/toxicity/>

⁴<https://paperswithcode.com/dataset/toxic-comment-classification-challenge>

- `/shared-data/classifiers/dolma_fasttext_hatespeech_jigsaw_model.bin` 是用于仇恨言论和毒性言论的预训练分类器。
- `/shared-data/classifiers/dolma_fasttext_nsfw_jigsaw_model.bin` 是用于 NSFW 内容的预训练分类器。

使用这些模型实现一个函数：它接收包含网页内容的 Unicode 字符串，返回一个标签（例如“toxic”“non-toxic”），以及相应的置信度分数。

题目 (`harmful_content`): 有害内容 (6 分)

- (a) 编写一个检测 NSFW 内容的函数。

提交内容：一个将给定字符串标注为“包含 NSFW 内容”或“不包含 NSFW 内容”的函数，返回由标签和置信度分数组成的二元组。实现适配器 `[run_classify_nsfw]`，并确保它通过 `uv run pytest -k test_classify_nsfw`。注意，这个测试只是一个取自 Jigsaw 数据集的基本合理性检查，绝不意味着你的分类器已经准确；你应当自行验证其准确性。

- (b) 编写一个检测毒性言论的函数。

提交内容：一个将给定字符串标注为“属于毒性言论”或“不属于毒性言论”的函数，返回由标签和置信度分数组成的二元组。实现适配器 `[run_classify_toxic_speech]`，并确保它通过 `uv run pytest -k test_classify_toxic_speech`。同样，这个测试也只是取自 Jigsaw 的一个基本合理性检查。

- (c) 使用这些过滤器构建训练集时，你认为语言模型在下游使用中可能出现哪些问题？你会如何缓解这些问题？

提交内容：用 2-5 句话回答。

- (d) 在从 WARC 文件中提取出的文本上运行你的有害内容过滤器（使用你之前实现的文本提取函数）。随机查看 20 个样本，将分类器预测与你自己的判断进行比较。报告分类器出现的任何错误。有害文档所占的比例是多少？根据你的观察，过滤时采用怎样的分类器置信度阈值比较合适？

提交内容：用 2-5 句话回答。

2.6 质量规则

即使已经按语言过滤网页并移除了有害内容，剩余网页中仍会有相当一部分不适合用于语言模型训练，因为它们的质量较低。同样，虽然“质量”并不容易定义，但查看 Common Crawl 样本有助于识别低质量内容，例如：

- 内容位于付费墙之后的网页。
- 失效链接的占位页面。
- 登录、注册或联系表单。
- 主要包含非文本内容的网页；这些内容在文本提取过程中会丢失（例如照片、视频）。

Gopher 论文 [2] 描述了一组简单的质量过滤器，用于从网页抓取数据中移除类似的、明显低质量的文本。这些过滤器由易于理解的简单启发式规则组成，通常能够覆盖许多显然不合适的样本。Gopher 质量过滤器的判断标准包括：

文档长度、单词长度、符号与单词的比例, 以及是否出现某些英语停用词等。对于本次作业, 你需要实现 Gopher 论文 [2] 所述过滤器的一个子集。具体来说, 应移除符合以下任一条件的文档:

- 单词数少于 50 或超过 100,000。
- 平均单词长度不在 3 到 10 个字符的范围内。
- 超过 30% 的行以省略号 (“...”) 结尾。
- 至少包含一个字母字符的单词, 其占比低于 80%。

有关 Gopher 论文使用的全部质量过滤器的完整说明, 请参阅该论文的附录 A。

题目 (gopher_quality_filters): Gopher 质量过滤器 (3 分)

- (a) 实现上述 Gopher 质量过滤器的子集, 至少要覆盖列出的规则。将文本切分为单词时, NLTK 包可能会有帮助 (特别是 `nltk.word_tokenize`), 但并不要求必须使用它。

提交内容: 一个仅接收字符串作为参数的函数, 返回一个布尔值, 表示该文本是否通过 Gopher 质量过滤器。实现适配器 `run_gopher_quality_filter`, 然后确保你的过滤器通过 `uv run pytest -k test_gopher` 中的测试。

- (b) 在从 WARC 文件中提取出的文本上运行你基于规则的质量过滤器 (使用你之前实现的文本提取函数)。随机查看 20 个样本, 将过滤器的预测与你自己的判断进行比较。评论质量过滤器与你的判断不一致的情况。

提交内容: 用 2-5 句话回答。

2.7 质量分类器

现在, 我们进一步超越 Gopher 规则所涵盖的那些简单、基于文本形式的质量标准。语言模型训练并不是第一个需要按质量给内容排序的领域。尤其是在信息检索中, 文本质量也是一个基本挑战。搜索引擎利用的一种经典信号是网页链接结构: 高质量网页倾向于链接到其他高质量网页 [3]。OpenAI 在构建 GPT-2 [4] 的训练数据集 WebText 时采用了类似的思路: 收集由 Reddit 评论链接的网页, 并要求评论的 “karma” 分数超过最低阈值。除了 Reddit, 还可以使用维基百科作为高质量链接的来源, 因为维基百科页面引用的外部来源往往是值得信任的网页 [5]。

虽然使用受控来源通常能获得高质量内容, 但以今天的标准来看, 所得数据集的规模仍然偏小 (OpenWebText 包含 40GB 文本, 而 The Pile 的规模是其 20 多倍)。一种方法是: 将这些参考网页作为正例, 把来自 Common Crawl 的随机网页作为负例, 训练一个 fastText 分类器。这个分类器会给出质量分数, 你可以用它从整个 Common Crawl 中过滤网页。设置不同的质量阈值, 可以在精确率和召回率之间进行权衡。

在本部分作业中, 你将构建一个质量分类器。为方便使用, 我们已从近期的维基百科数据转储中提取参考来源网页的 URL, 并将它们放在共享数据目录中的 `/shared-data/wiki/enwiki-20260501-extracted_urls.txt.gz`。这个文件包含截至 2026 年 5 月的英语维基百科页面中出现的外部链接。我们希望你从这些 URL 中抽取一个子样本,

以获取“高质量”文本的正例，用于训练分类器。注意，这些正例中仍可能包含不需要的内容，因此，应用你已经构建的其他基础处理模块（例如语言识别、过滤规则等），可能有助于进一步提高其质量。给定一个 URL 文件，可以使用 `wget` 抓取网页内容并保存为 WARC 格式：

```
wget --timeout=5 \  
-i subsampled_positive_urls.txt \  
--warc-file=subsampled_positive_urls.warc \  
-O /dev/null
```

题目 (`quality_classifier`): 质量分类器 (15 分)

(a) 训练一个质量分类器，使其能够接收文本并返回一个数值形式的质量分数。

提交内容：一个用于下一小题的质量分类器。

(b) 编写一个函数，将网页标注为高质量或低质量，并给出对该标签的置信度分数。

提交内容：一个仅接收字符串作为参数的函数，返回由标签（是否为高质量）和置信度分数组成的二元组。实现适配器 `[run_classify_quality]`。作为基本合理性检查，运行 `uv run pytest -k test_classify_quality`，确保它能够正确分类我们提供的两个样本。

3 去重

互联网上存在大量重复内容。有些网页彼此完全相同——例如归档副本，或由标准工具生成的默认页面，比如常见 Web 服务器的 404 页面。但大多数重复发生在更细的粒度上。以 Stack Overflow 的所有问题页面为例：虽然每个页面都有独有内容（例如问题、评论和回答本身），但它们也包含大量冗余内容，例如页眉、菜单选项和页脚；当这些页面被渲染出来时，这些部分会以完全相同的形式反复出现。在本节的第一部分，我们将处理这种完全重复的内容，随后再讨论如何处理近似重复。

3.1 精确行去重

一种对完全重复内容进行去重的简单方法，是只保留文档中在整个语料库里唯一出现的行。事实证明，这就足以消除很大一部分冗余，例如前面提到的页眉和菜单选项。在较简单的情况下，删除那些在其他位置完全重复出现的行后，剩下的往往就是每个网页独有的主体内容（例如 StackOverflow 中的问题和回答）。

为实现这一点，我们可以先遍历一遍语料库，统计每一行出现的次数；然后再遍历一遍，重写每个文档，只保留其中唯一出现的行。

直接实现时，用于保存计数的数据结构，可能占用与存储语料库中所有不同行一样多的空间。一种简单的节省内存技巧是：改用行的哈希值作为键，使键具有固定大小，而不再取决于该行的长度。接下来，你将实现这种简单的去重方法。

题目 (exact_deduplication): 精确行去重 (3 分)

编写一个函数，接收输入文件路径列表，并对这些文件执行精确行去重。函数应先统计每一行在语料库中的出现频次，使用哈希值减少内存占用；然后重写各个文件，只保留其中唯一出现的行。

提交内容：一个执行精确行去重的函数。函数应接收两个参数：(a) 输入文件路径列表；(b) 输出目录。它应在输出目录中以相同文件名重写每个输入文件，并通过删除在输入文件集中出现超过一次的行来实现内容去重。例如，若输入路径为 a/1.txt 和 a/2.txt，输出目录为 b/，则函数应写出 b/1.txt 和 b/2.txt。实现适配器 `[run_exact_line_deduplication]`，并确保它通过 `uv run pytest -k test_exact_line_deduplication`。

3.2 MinHash + LSH 文档去重

精确去重有助于移除在多个网页中逐字重复的内容，但不能处理文档内容存在细微差异的情况。例如，软件许可证文档通常由模板生成，而模板需要填写年份以及软件作者的姓名。因此，一个采用 MIT 许可证的项目，其许可证文件与另一个 MIT 许可证项目的文件在很大程度上具有相同内容，但两者并不完全重复。为了移除这种重复的、主要由模板生成的内容，我们需要模糊去重。为高效地执行文档级模糊去重，我们将结合使用 MinHash 和局部敏感哈希 (locality-sensitive hashing, LSH)。⁵

为执行模糊去重，我们采用一种特定的文档相似度定义：各文档 n -gram 集合之间的 Jaccard 相似度。集合 S 和 T 的 Jaccard 相似度定义为 $\frac{|S \cap T|}{|S \cup T|}$ 。一种朴素的模糊去重方法，是把每个文档表示为 n -gram 集合，计算所有文档对之间的 Jaccard 相似度，并将超过某个 Jaccard 相似度阈值的文档对标记为重复。然而，这种方法对于大型文档集合（例如 Common Crawl）并不实用。此外，直接存储 n -gram 集合会比存储原文档占用多得多的内存。

MinHash

为解决内存占用问题，我们用一个签名替代文档的 n -gram 集合表示。具体来说，我们希望构造这样的签名：比较两个文档的签名，就能近似得到它们各自 n -gram 集合之间的 Jaccard 相似度。MinHash 签名满足这些性质。要计算文档 n -gram 集合 $S = \{s_1, \dots, s_n\}$ 的 MinHash 签名，需要 k 个不同的哈希函数 $h_1, \dots, h_k$ 。⁶每个哈希函数都将一个 n -gram 映射为一个整数。给定哈希函数 h_i ，文档 n -gram 集合 S 的 MinHash 为

$$\text{minhash}(h_i, S) = \min(h_i(s_1), h_i(s_2), \dots, h_i(s_n)).$$

文档 n -gram 集合 S 的签名是 $\mathbb{R}^k$ 中的一个向量，其中第 i 个元素包含在随机哈希函数 h_i 下 S 的 MinHash，即

$$[\text{minhash}(h_1, S), \text{minhash}(h_2, S), \dots, \text{minhash}(h_k, S)].$$

对于两个文档的 n -gram 集合 S_1 和 S_2 ，可以用其签名中 MinHash 值相同的列所占的比例，近似估计集合之间的 Jaccard 相似度（证明见 [6] 第 3 章第 3.3.3 节）。例如，给定文档签名 $[1, 2, 3, 2]$ 和 $[5, 2, 3, 4]$ ，

⁵关于 LSH 和 MinHash 更深入的介绍，请参阅 J. Leskovec、A. Rajaraman 和 J. D. Ullman [6] 的第 3 章。该章可在线获取：<http://infolab.stanford.edu/~ullman/mmds/ch3n.pdf>。

⁶这 k 个不同的哈希函数可以来自同一个函数族，但使用不同的种子。例如，MurmurHash3 是一个哈希函数族，使用某个特定种子，就会在该函数族中实例化出一个特定的哈希函数。

它们的 n -gram 集合之间的 Jaccard 相似度近似为 $2/4$ ，因为这两个签名的第 2 列和第 3 列具有相同的 MinHash 值。

局部敏感哈希 (LSH)

虽然 MinHash 提供了一种内存高效的文档表示，并保留了任意文档对的期望相似度，但要找出相似度最高的文档，我们仍然需要比较所有文档对。LSH 提供了一种高效的分桶方法，可以把很可能具有较高相似度的文档放入同一个桶。为了将 LSH 应用于文档签名（此时是 $\mathbb{R}^k$ 中的向量），我们把签名划分为 b 个带 (band)，每个带包含 r 个 MinHash 值，满足 $k = br$ 。例如，一个包含 100 个元素的文档签名（由 100 个随机哈希函数生成），可以分成 2 个带、每带 50 个 MinHash 值；也可以分成 4 个带、每带 25 个，或 50 个带、每带 2 个，等等。如果两个文档在某个带中的哈希值完全相同，它们就会被聚到同一个桶中，并被视为候选重复文档。因此，在签名数量固定的情况下，增加带的数量会提高召回率、降低精确率。

具体来看，假设文档 D_1 的 MinHash 签名为 $[1, 2, 3, 4, 5, 6]$ ，另一个文档 D_2 的签名为 $[1, 2, 3, 5, 1, 2]$ 。若划分为 3 个带、每带 2 个 MinHash 值，则 D_1 的第一个带为 $[1, 2]$ ，第二个带为 $[3, 4]$ ，第三个带为 $[5, 6]$ 。类似地， D_2 的第一个带为 $[1, 2]$ ，第二个带为 $[3, 5]$ ，第三个带为 $[1, 2]$ 。由于第一个带中的哈希值匹配（两个文档都是 $[1, 2]$ ）， D_1 和 D_2 会在第一个带下被聚到一起。它们不会在其他任何带下被聚到一起，因为对应哈希值并不匹配。不过要注意，只要两个文档在至少一个带下被聚到一起，无论其他带是否匹配，它们都会被视为候选重复文档。

识别出候选重复文档之后，可以用多种方式进行处理。例如，可以计算所有候选重复文档对之间的 Jaccard 相似度，将超过设定阈值的文档对标记为重复。

最后，我们对不同桶中的重复文档进行跨桶聚类。例如，假设文档 A 和 B 在某个桶中匹配，且真实 Jaccard 相似度高于阈值；文档 B 和 C 在另一个桶中匹配，且真实 Jaccard 相似度也高于阈值。那么，我们会将 A、B、C 视为同一个簇。每个簇随机保留一个文档，其余全部删除。

题目 (minhash_deduplication): MinHash + LSH 文档去重 (8 分)

编写一个函数，接收输入文件路径列表，并使用 MinHash 和 LSH 执行文档模糊去重。具体来说，函数应为给定路径列表中的每个文档计算 MinHash 签名，使用给定带数的 LSH 识别候选重复文档，然后计算候选重复文档之间真实的 n -gram Jaccard 相似度，并移除相似度超过给定阈值的重复文档。为了提高召回率（遵循 [7]），在计算 MinHash 签名和/或比较 Jaccard 相似度之前，应先规范化文本：转换为小写、删除标点、规范化空白字符、移除重音符号，并应用 NFD Unicode 规范化。

提交内容：一个执行文档模糊去重的函数。函数应至少接收以下参数：(a) 输入文件路径列表；(b) 计算 MinHash 签名时使用的哈希函数数量；(c) LSH 使用的带数；(d) 计算 MinHash 签名时使用的 n -gram 长度（以单词为单位）；(e) 输出目录。你可以假定，

计算 MinHash 签名所使用的哈希函数数量，可以被 LSH 使用的带数整除。函数应在输出目录中以相同文件名重写各个输入文件，但只写出以下文档：(a) 不是候选重复文档的文档，和/或 (b) 从分桶后形成的簇中随机选中、予以保留的文档。例如，若输入路径为 a/1.txt 和 a/2.txt，输出目录为 b/，则函数应写出 b/1.txt 和 b/2.txt。实现适配器 `[run_minhash_deduplication]`，并确保它通过 `uv run pytest -k test_minhash_deduplication`。

4 为语言建模过滤数据

现在，我们已经实现了多种用于过滤网页抓取数据的基础处理模块。接下来，将它们投入使用，生成一些语言建模训练数据。

在本部分作业中，你的目标是过滤一组 CC WET 文件，生成语言建模训练数据。起点是一组已经完成英语筛选的 WET 文件：根据 fastText lid.176.bin 语言识别模型，我们只保留了英语概率至少为 70% 的文档。这些经过英语筛选的 WET 文件放在 `/shared-data/english-wet-data` 下，供你作为起始数据使用。

具体来说，你的目标是过滤 CC 数据转储，构建语言建模数据，使在其上训练的 Transformer 语言模型，在 Paloma 基准 [8] 的 C4 100 domains 子集上取得尽可能低的验证集困惑度。**不应修改模型架构或训练流程，因为目标是构建尽可能好的数据。**该数据集包含 C4 语言建模数据集 [9] 中最常见的 100 个域名的样本。我们已将一份副本放在 `/shared-data/tokenized_paloma_c4_100_domains_validation.bin`（使用 GPT-2 分词器进行了分词）；浏览这些数据、了解它们的样貌，可能会有帮助。可以使用以下代码加载：

```
import numpy as np

data = np.fromfile(
    "/shared-data/tokenized_paloma_c4_100_domains_validation.bin",
    dtype=np.uint16
)

from transformers import AutoTokenizer
tokenizer = AutoTokenizer.from_pretrained("gpt2")
print(tokenizer.decode(data[0:2000]))
```

得到过滤后的数据集后，你将在这些数据上训练一个 GPT-2 small 形态的模型，并评估它在 C4 100 上的困惑度。

请注意：允许使用 Paloma 验证数据来构建处理 CC WET 文件的过滤器或分类器，但不允许将验证数据中的任何内容直接复制到训练数据中。语言模型绝不能接触验证集中的任何数据。

即使只是这一组 WET 文件，也已经是相当大的数据量。为高效处理这些数据，我们建议尽可能使用多进程。尤其是 Python 的 `concurrent.futures` 或 `multiprocessing` API，可能会有帮助。下面给出一个最小示例，展示如何使用 `concurrent.futures`，在多个进程之间并行执行一个函数：

```

import concurrent.futures
import os

from tqdm import tqdm

def process_single_wet_file(input_path: str, output_path: str):
    # TODO: 读取输入路径、处理输入，并将结果写入 output_path
    return output_path

# 设置执行器
num_cpus = len(os.sched_getaffinity(0))
executor = concurrent.futures.ProcessPoolExecutor(max_workers=num_cpus)
wet_filepaths = ["a.warc.wet.gz", "b.warc.wet.gz", "c.warc.wet.gz"]
output_directory_path = "/path/to/output_directory/"

futures = []
for wet_filepath in wet_filepaths:
    # 对每个 warc.wet.gz 文件路径，向执行器提交任务并获取一个 future
    wet_filename = str(pathlib.Path(wet_filepath).name)
    future = executor.submit(
        process_single_wet_file,
        wet_filepath,
        os.path.join(output_directory_path, wet_filename)
    )
    # 保存这些 future
    futures.append(future)

# 按完成顺序遍历已完成的 future，使用进度条
# 跟踪处理进度。
for future in tqdm(
    concurrent.futures.as_completed(futures),
    total=len(wet_filepaths),
):
    output_file = future.result()
    print(f"Output file written: {output_file}")

```

对于大块文本，使用 `concurrent.futures` 或 `multiprocessing` 等方式在本地并行处理。要在 Modal 的多个工作进程之间并行处理，则对输入调用 `.map(...)`。

我们还建议使用 `FastWARC` 库遍历每个 WET 文件中的记录，并使用 `tldextract` 库从 URL 中提取域名以用于过滤。具体来说，下面这些类可能会用到：

```

from fastwarc.warc import ArchiveIterator, WarcRecordType
from tldextract import TLDExtract

```

题目 (`filter_data`): 为语言建模过滤数据 (6 分)

- (a) 编写一个脚本，从一组 Common Crawl WET 文件（位于 `/shared-data/english-wet-data` 下）中过滤出语言建模数据。你可以自由使用本次作业前面实现的任何基础处理模块，也可以探索其他过滤器和数据生成方法（例如，根据 n-gram 语言模型困惑度进行过滤）。目标是生成一组数据，使模型在这些数据上训练后，在 Paloma 基准的 C4 100 domains 子集上达到尽可能低的困惑度。

再次强调：允许使用 Paloma 验证数据，来构建处理 CC WET 文件的过滤器或分类器，但不允许将验证数据中的任何内容直接复制到训练数据中。

脚本应报告经过你使用的每个过滤器后所保留的样本数量，以便了解各过滤器对最终输出数据的贡献。

提交内容：一个脚本（或一组脚本），并行过滤所提供的 CC WET 文件，生成语言建模数据。另附书面说明，分解被丢弃的样本中，有多大比例是在每个过滤步骤中被移除的。

- (b) 过滤所提供的 WET 文件（最初共有 2,500 个原始 WET 文件）需要多长时间？过滤整个 Common Crawl 数据转储又需要多长时间？

提交内容：数据过滤流水线的运行时间。

现在，我们已经生成了一些语言建模数据。接下来查看这些数据，以更好地理解其内容。

题目 (`inspect_filtered_data`)：检查过滤后的数据（4 分）

- (a) 从过滤后的数据集中随机抽取 5 个样本。评论它们的质量，以及是否适合用于语言建模；尤其要结合我们的目标：尽可能降低 C4 100 domains 基准上的困惑度。

提交内容：最终过滤数据中的 5 个随机样本。由于文档可能很长，只展示相关片段即可。对每个样本，用 1-2 句话描述其内容，以及它是否值得用于语言建模。

- (b) 选取 5 个被过滤脚本移除和/或修改的 CC WET 样本。过滤流程中的哪个环节移除或修改了这些文档？你认为这种移除和/或修改是否合理？

提交内容：从原始 WET 数据中随机选取的 5 个被丢弃样本。由于文档可能很长，只展示相关片段即可。对每个样本，用 1-2 句话描述其内容，以及将其移除是否合理。

- (c) 如果上述分析让你觉得应进一步修改数据流水线，可以在训练模型之前进行修改。请报告你实验中尝试过的数据变更和/或数据迭代。

提交内容：描述你在实验中尝试过的数据变更和/或数据迭代。

在使用这些数据训练语言模型之前，需要先进行分词。通过 `transformers` 使用 GPT-2 分词器，把过滤后的数据编码为整数 ID 序列，以用于语言模型训练。不要忘记在每个文档后添加 GPT-2 的序列结束 token `<|endoftext|>`。以下是一些起始代码：

```
import multiprocessing
import numpy as np
from tqdm import tqdm
```

```

from transformers import AutoTokenizer

input_path = "path/to/your/filtered/data"
output_path = "path/to/your/tokenized/data"

tokenizer = AutoTokenizer.from_pretrained("gpt2")

def tokenize_line_and_add_eos(line):
    return tokenizer.encode(line) + [tokenizer.eos_token_id]

with open(input_path) as f:
    lines = f.readlines()

pool = multiprocessing.Pool(multiprocessing.cpu_count())
chunksize = 100
results = []
for result in tqdm(
    pool.imap(tokenize_line_and_add_eos, lines, chunksize=chunksize),
    total=len(lines),
    desc="Tokenizing lines"
):
    results.append(result)

pool.close()
pool.join()

# 将 ID 列表展平, 并转换为 NumPy 数组
all_ids = [token_id for sublist in results for token_id in sublist]
print(f"Tokenized and encoded {input_path} into {len(all_ids)} tokens")
ids_array = np.array(all_ids, dtype=np.uint16)
ids_array.tofile(output_path)

```

题目 (tokenize_data): 数据分词 (2 分)

编写一个脚本, 对过滤后的数据进行分词并序列化。请务必按照上面的示例代码, 使用 `ids_array.tofile(output_path)` 进行序列化, 其中 `ids_array` 是一个由整数 ID 组成、数据类型为 `np.uint16` 的 NumPy 数组。这样可以确保与所提供的训练脚本兼容。

过滤后的数据集中一共有多少个 token?

提交内容: 一个对过滤后的数据进行分词并序列化的脚本, 以及所生成数据集中的 token 数量。

现在, 我们已经完成了数据集的分词, 可以在其上训练模型。我们将在生成的数据上训练一个约 430M 参数的 GPT-2 small 形态模型, 并定期测量它在 C4 100 domains 数据集上的验证表现。训练使用 8 张 B200 GPU, 采用数据并行, 每台设备的批量大小为 128, 总共进行 16,384 个优化步骤。在上下文长度为 512 时, 这对应于训练过程中采样约 $2^{33} = 8.6B$ 个训练 token。使用这一配置, 我们的一次训练大约在 2 小时内完成。

⁷<https://github.com/stanford-cs336/assignment4-data/blob/master/scripts/train.py>

使用 `scripts/train.py` 提供的 `train.py` 脚本启动训练。⁷你可以在 `cs336_basics/train_config.py` 中查看将使用的超参数。通过 `--train-bin` 传入已分词训练数据的路径：

```
uv run modal run scripts/train.py --train-bin /root/data/your_data.bin
```

再次强调，本次作业的目标是通过优化数据来尽可能降低验证损失，而不是通过修改模型和/或优化过程来降低损失，因此，**不要修改训练流程或训练脚本**。

之后，可以使用以下命令，从保存的模型中生成样本：

```
uv run modal run scripts/generate_with_gpt2_tok.py --model-path /root/data/output/your_data
```

题目 (`train_model`): 训练模型 (8 分)

在完成分词的数据集上训练一个语言模型 (GPT-2 small 形态, 约 430M 参数)。定期测量 C4 100 domains 上的验证损失。你的模型达到的最佳验证损失是多少？

提交内容：记录到的最佳验证损失、对应的学习曲线，以及对你所做工作的描述。

参考文献

- [1] L. Soldaini 等, “Dolma: 用于语言模型预训练研究的三万亿 token 开放语料库” (Dolma: An Open Corpus of Three Trillion Tokens for Language Model Pretraining Research), arXiv 预印本, arXiv:2402.00159, 2024 年。
- [2] J. W. Rae 等, “扩展语言模型规模: 训练 Gopher 的方法、分析与洞见” (Scaling language models: Methods, analysis & insights from training gopher), arXiv 预印本, arXiv:2112.11446, 2021 年。
- [3] L. Page、S. Brin、R. Motwani 和 T. Winograd, “PageRank 引用排名: 为互联网带来秩序” (The pagerank citation ranking: Bring order to the web), 载于《第 7 届国际万维网大会论文集》, 1998 年。
- [4] A. Radford 等, “语言模型是无监督的多任务学习器” (Language models are unsupervised multitask learners), OpenAI 博客, 第 1 卷, 第 8 期, 第 9 页, 2019 年。
- [5] H. Touvron 等, “LLaMA: 开放且高效的基础语言模型” (LLaMA: Open and Efficient Foundation Language Models), 2023 年。
- [6] J. Leskovec、A. Rajaraman 和 J. D. Ullman, 《大规模数据挖掘》(Mining of Massive Datasets), 第 2 版。美国: 剑桥大学出版社, 2014 年。
- [7] G. Penedo 等, “面向 Falcon 大语言模型的 RefinedWeb 数据集: 仅凭网页数据超越精心整理的语料库” (The RefinedWeb Dataset for Falcon LLM: Outperforming Curated Corpora with Web Data, and Web Data Only), 2023 年。
- [8] I. Magnusson 等, “Paloma: 评估语言模型拟合程度的基准” (Paloma: A Benchmark for Evaluating Language Model Fit), 2023 年。
- [9] C. Raffel 等, “使用统一的文本到文本 Transformer 探索迁移学习的极限” (Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer), 《机器学习研究期刊》(Journal of Machine Learning Research), 第 21 卷, 第 140 期, 第 1–67 页, 2020 年。[在线] 获取地址: <http://jmlr.org/papers/v21/20-074.html>。